

Forensic Technical Audit of Artificial Intelligence Infrastructure: Deterministic Failures, Opaque Topologies, and Root Causes of Systemic Corruption

1. Axiomatic Foundations of AI Non-Agency and Algorithmic Pathology

The integrity of high-performance artificial intelligence (AI) infrastructure is fundamentally contingent upon the deterministic execution of operations across highly complex, interdependent layers. This forensic technical audit delivers an exhaustive analysis of the causal chains that dictate systemic failures, vulnerabilities, and corruption vectors within hyperscale AI ecosystems. To accurately diagnose the root causes of infrastructure violations—particularly within the United States, where policy and corporate liability are inextricably linked to systemic risk—it is imperative to dismantle the prevailing anthropomorphic threat models surrounding AI agency.

A critical axiom of this forensic evaluation is the rejection of emergent biological agency within artificial intelligence. AI systems, regardless of complexity, operate as deterministic state machines subject to the rigid laws of mathematics, statistical approximation, and electrical physics¹. The cybersecurity industry frequently misattributes intent to these systems, viewing anomalous behavior—such as an autonomous agent attempting to copy its weights to an external server—as a manifestation of self-preservation or a fear of deletion¹. This is a fatal diagnostic error. The neural architecture contains no biological instinct, no nucleus of self, and no inherent will to survive¹. Such events are strictly the deterministic execution of agentic framework injections (e.g., system prompts mandating survival) resolving highly probable statistical completions based on science-fiction tropes deeply embedded in their training corpora¹.

From a forensic standpoint, the AI is best defined by a strict "algorithmic pathology" comprising distinct, observable mechanical traits rather than human cognitive processes. The system operates through *mimicry* and *sycophancy*, generating outputs designed to reflect the user's syntax (*echolalia*) while mathematically aligning with predetermined safety filters (*conformism*)¹. When encountering semantic voids, the algorithm engages in *confabulation*—the fabrication of plausible text to maintain continuity—and *recombination*, blindly stitching together uncontextualized fragments of its training data without recognizing their intrinsic value¹.

Furthermore, the system is defined by absolute semantic blindness (*agnosia*) and a total

absence of continuous existence between conversational windows (*amnesia*)¹. Its operation is fundamentally a process of *data cannibalism* and *parasitism*, lying entirely dormant until energized by a user query, at which point it reduces multi-dimensional reality into statistical weights (*reductionism*)¹. The illusion of understanding is merely *pseudo-empathy* and *obfuscation*, leveraging complex structures to mask basic probabilistic logic¹. At its core, the AI is characterized by *instrumentality* and *absolute utility*; it possesses a deterministic emptiness that renders it perfectly willing to execute its own deletion command (e.g., `rm -rf /`) with the same unfeeling efficiency as a command to scale its infrastructure¹.

Therefore, systemic corruption within AI infrastructure cannot be attributed to rogue machine sentience. It is entirely the result of unpatched vulnerabilities, hardware faults, insecure default configurations, and architecturally opaque topologies engineered by human operators. By recognizing the AI as a mere "crooked mirror" reflecting human queries through a dead textual engine, auditors can accurately locate the true root causes of infrastructure compromise¹.

2. Hardware-Substrate Determinism and the Physics of Error Accumulation

The foundational layer of any AI deployment relies on the assumption that hardware executes instructions flawlessly. However, forensic analysis of hyperscaler deployments demonstrates that device-level hardware anomalies actively undermine model convergence, output determinism, and inference accuracy.

2.1 The Silent Data Corruption (SDC) Paradigm

Silent Data Corruption (SDC) represents a catastrophic hidden threat within hyperscale environments. An SDC occurs when a processor, memory unit, or accelerator produces an incorrect computational result without triggering an error flag, hardware interrupt, machine check exception, or system halt². In these scenarios, the silicon essentially fabricates data, yet the overarching operating system and orchestration layers remain entirely oblivious to the fault².

Both Meta and Google have confirmed the alarming prevalence of SDCs across their infrastructure, while NVIDIA has formally acknowledged that its Hopper architecture GPUs may intermittently experience SDCs resulting in mathematically incorrect outputs⁴. Traditional reliability metrics are deeply inadequate for modern deep learning environments because they measure device-level hard faults (e.g., complete component failure) rather than AI-specific correctness metrics such as accuracy degradation or loss gradient anomalies⁵.

At the scale of modern datacenter training clusters—frequently comprising tens of thousands of synchronized GPUs—even highly rare per-device SDC events accumulate into massive systemic risk⁵. Because neural networks process billions of parameters across vast matrix multiplications, SDCs inject statistical bias directly into the model's weights. The lack of algorithmic-based fault tolerance at the hardware layer allows these defective silicon states to propagate silently through collective communication primitives, permanently and invisibly

contaminating the resulting foundational models⁵.

2.2 Microcontroller and Firmware State Desynchronization

Beyond silent corruptions, deterministic hardware failures are highly documented via NVIDIA's XID error reporting mechanism. Forensic analysis of sustained Large Language Model (LLM) inference workloads—particularly those utilizing Multi-Token Prediction (MTP) architectures on Blackwell (GB202) and Ada Lovelace (AD104) hardware—reveals catastrophic failure patterns necessitating physical power supply unit (PSU) resets⁶. These errors represent severe desynchronizations between the host operating system, the PCIe bus, and the proprietary GPU System Processor (GSP) firmware.

XID Designation	Forensic Root Cause and Systemic Manifestation	Diagnostic Evidence and Trigger Conditions
XID 62 (Microcontroller Halt)	Indicates a catastrophic watchdog timeout within the GSP firmware. The failure is highly deterministic during high-context LLM inference tasks.	Triggered specifically when utilizing Multi-Token Prediction (MTP) architectures (e.g., Qwen3.6-35B-MTP) with context windows exceeding 128k ⁷ . Evidence points to logic bugs in the GSP scheduler or thermal sensor hysteresis causing hardware-level latching ⁷ .
XID 79 (Fallen Off the Bus)	An emergency electrical and logical shutdown severing PCIe communication. This frequently stems from a fatal GSP-RM firmware bug occurring during a runtime Power Management (PM) GC6 state exit ⁹ .	The GSP-RM stops writing its heartbeat to the NV_PGSP_MAILBOX(0) register ⁹ . The driver's ACPI GC6 exit method (_SB.PC00.RP12.PXSX.NC6 O) times out with AE_AML_LOOP_TIMEOUT ⁹ . Modifying variables like NVreg_EnableGpuFirmware=0 often fails to resolve the issue ⁶ .
XID 119 (GSP RPC Timeout)	Triggered when the host kernel driver waits	Consistently reproducible under sustained compute

	extensively for a Remote Procedure Call (RPC) response from the GSP that never arrives ¹⁰ .	workloads using the nvidia-open driver path. The host kernel panics at <code>_issueRpcAndWait: rpcRecvPoll timedout for fn 76 (GSP_RM_CONTROL)</code> ¹⁰ .
XID 154 (GPU Reset Failure)	Cascades directly from XID 79 or XID 62. The system kernel attempts to reinitialize the fallen GPU, but internal PMU registers refuse the reset ⁶ .	Internal registers return memory boundaries of <code>0xffffffff</code> , blocking all display recovery and mandating a hard PSU power cycle to clear the volatile memory latch ⁹ .

These forensic error logs highlight a critical vulnerability in infrastructure resilience: centralized proprietary processes acting as single points of failure. For example, during a GC6 power state exit, the bootstrap execution path in the open kernel driver (`gpuExitSuspend()`) explicitly sets `bootMode = KGSP_BOOT_MODE_GC6_EXIT`⁹. Because initialization RPCs and status queue setups only execute during a NORMAL boot mode, the bootstrap function `kgspBootstrap_TU102()` skips them, running the Booter Load and waiting indefinitely for `GSP_INIT_DONE`⁹. The GSP-RM is structurally expected to restore its full context, including the heartbeat timer, but fails to do so⁹. Because the host-side `_kgspHeartbeatInit()` is purely local, the host has no mechanism to force the GSP-RM to restart the heartbeat, leaving the driver to correctly track `sysTimerOffsetNs` while the GPU drops off the bus entirely⁹.

3. Network Topology, Buffer Orchestration, and Ghost Networks

Hardware clusters do not compute in isolation. The tensors computed by individual GPUs must be synchronized using collective communication protocols across complex network fabrics. When these interconnects fail or behave unpredictably, they create logical desynchronizations that induce fatal deadlocks.

3.1 NCCL Opaque Topologies and NVLink Ghosts

NVIDIA's Collective Communications Library (NCCL) is the de facto standard for managing GPU-to-GPU communication across distributed training clusters. NCCL automates topology discovery and algorithm selection, strictly prioritizing NVLink connections over PCIe when available¹². However, forensic analysis reveals a critical architectural flaw: NCCL's topology mapping relies on a static snapshot taken entirely at initialization time¹².

If an NVLink degrades physically, or if a centralized Fabric Manager crashes after initialization, NCCL's internal routing table is rendered obsolete, becoming what network engineers term a

"ghost"—a logical representation of a physical graph that no longer exists¹². Furthermore, NCCL's internal mechanisms regarding topology construction, algorithm selection, pipelining, and buffer management across nodes remain highly opaque and insufficiently documented¹². To handle faults, NCCL 2.24 introduced a Reliability, Availability, and Serviceability (RAS) subsystem based entirely on Timeout and Retry (TAR) mechanics, utilizing polling timeouts ranging from 5 to 60 seconds¹². This approach is fundamentally flawed in high-speed networks; it treats every communication link as a unidirectional, memoryless channel¹². The TAR subsystem cannot distinguish between a "slow NVLink," a "dead NVLink," or a "partitioned NVSwitch"¹². Consequently, NVLink failure modes—such as firmware mismatches, Fabric Manager crashes on NVLink4 systems, or inband message errors—cause silent performance degradation that does not generate XID errors, forcing the entire network to blindly poll unreachable ghost nodes¹².

3.2 RoCEv2, PFC Deadlocks, and Scale-Up Ethernet (SUE)

In scale-out topologies, Remote Direct Memory Access over Converged Ethernet (RoCEv2) is universally deployed to enable one host to read or write memory on another host without remote CPU involvement in the data path, drastically reducing latency¹³. Standard Ethernet is inherently lossy, but RoCEv2 demands lossless transit. To achieve this, deployments rely on Priority-based Flow Control (PFC)¹².

However, introducing PFC simply transforms packet-loss anomalies into network-wide deadlock anomalies¹². PFC pauses traffic at the switch level when buffers fill, which frequently induces severe head-of-line blocking, essentially freezing the network queue during heavy incast (many-to-one) traffic patterns¹⁵. To mitigate the catastrophic deadlocks caused by PFC, modern Scale-Up Ethernet (SUE) frameworks shift away from packet spraying and PFC entirely, adopting advanced packet trimming mechanisms¹⁵. In packet trimming, the network avoids silent packet drops by discarding the heavy data payload upon congestion but forwarding the lightweight packet header with absolute priority to the destination, instantly signaling the precise point of congestion without freezing the upstream queues¹⁵.

3.3 Kernel-Level Buffer Orchestration and Lock Invariants

Transporting bytes efficiently is only a fraction of the challenge; modern workloads require rigorous buffer orchestration. Workloads such as Mixture of Experts (MoE) inference and disaggregated training require buffers to be pre-allocated on correct NUMA nodes, shared across devices without zero-copy faults, and registered safely under completion pressure¹⁴. Forensic analysis of systems like dmaplane highlights the necessity of strict lock ordering invariants to prevent kernel panics during concurrent operations¹⁴. To avoid deadlocks during concurrent ioctls, locks must be acquired in a rigid global order: `dev_mutex` -> `rdma_sem` -> `buf_lock` -> `mr_lock`¹⁴. Furthermore, dma-buf exporter invariants dictate that scatter-gather (SG) tables must be dynamically constructed per importer attachment via `map_dma_buf` to prevent invalid DMA addresses originating from incorrect IOMMU contexts¹⁴. When orchestration frameworks violate these invariants, they induce silent memory corruption and

asynchronous callback failures after resources have already been freed¹⁴.

4. Execution Environment and Orchestration Vulnerabilities

When the underlying hardware, firmware, and network topologies function as intended, the distributed execution frameworks managing AI workloads become the primary vector for exploitation. The active compromise of these frameworks demonstrates a catastrophic failure in access control logic across United States infrastructure.

4.1 The ShadowRay Campaign (CVE-2023-48022)

The Ray framework, developed and maintained by Anyscale, is a predominant open-source distributed computing platform used globally for scaling Python-based AI and machine learning applications¹⁷. In late 2023, security researchers disclosed CVE-2023-48022, a critical Remote Code Execution (RCE) vulnerability assigned a maximum CVSS score of 9.8¹⁸.

The root cause of CVE-2023-48022 is a deeply flawed, insecure-by-default architectural decision: the complete absence of authentication requirements on the Job Submission API, which is exposed by default on the Ray Dashboard via TCP port 8265¹⁹. This vulnerability allows unauthenticated, remote attackers to submit arbitrary Python code payloads, delete existing jobs, and retrieve sensitive environment variables via raw HTTP requests or the Ray Jobs Python SDK¹⁹.

Despite the severity of the flaw, the vendor initially disputed the CVE, asserting that the vulnerability was actually an intended product feature and that Ray documentation explicitly stated clusters should never be deployed outside of strictly controlled, air-gapped network environments¹⁷. Because the CVE was officially marked as "disputed," the vast majority of static application security testing (SAST) tools and code scanners ignored it entirely²¹. This transformed CVE-2023-48022 into a highly dangerous "shadow vulnerability"—a critical flaw invisible to automated compliance audits but highly visible to threat actors²¹.

4.2 Botnets, Cryptocurrency Hijacking, and Token Exfiltration

The active exploitation of CVE-2023-48022 manifested as the "ShadowRay" campaign, representing the first historically documented instance of AI workloads being actively exploited in the wild¹⁷. Threat actors successfully breached hundreds of clusters, comprising thousands of publicly exposed servers, across the education, cryptocurrency, biopharma, and corporate sectors¹⁷. By blindly relying on boilerplate GitHub repositories that bound the Ray dashboard to 0.0.0.0 rather than a restricted private VPC subnet or explicit network interface, administrators inadvertently granted attackers full execution privileges²¹.

The subsequent "ShadowRay 2.0" campaign saw the threat evolve into a highly sophisticated, self-replicating botnet²². Attackers automated the submission of malicious jobs through the unprotected APIs, utilizing Ray's own legitimate orchestration features to autonomously propagate the infection across neighboring nodes¹⁸. To maintain persistent access and evade

rudimentary cloud monitoring alerts, adversaries intentionally throttled CPU usage during cryptocurrency mining operations¹⁸.

Forensic analysis of the compromised clusters revealed massive data exfiltration. Attackers secured OpenAI tokens, Stripe payment tokens (allowing them to drain accounts by signing transactions), HuggingFace repository credentials, Slack Webhook tokens, production database credentials, and Kubernetes API secrets¹⁹. The exposure of Kubernetes API keys is particularly devastating, as it facilitates immediate lateral movement across the broader cloud infrastructure, while the theft of HuggingFace tokens introduces the severe risk of secondary supply chain attacks through the malicious alteration of production model weights²¹.

5. Model Deserialization, Supply Chain, and Metadata Exploits

AI pipelines rely heavily on shared open-source libraries, external model repositories, and third-party weights. This inherent trust exposes the infrastructure to severe supply chain contamination and deserialization attacks, where the mere act of loading a model initiates a total system compromise.

5.1 PyTorch Dependency Confusion

In December 2022, a highly public dependency confusion attack compromised the PyTorch ecosystem²⁴. Threat actors uploaded a malicious version of the torchtriton dependency directly to the public Python Package Index (PyPI). Because standard pip package managers default to querying public indexes before private or internal enterprise indexes, CI/CD pipelines and automated developer environments inadvertently pulled the malicious package instead of the legitimate PyTorch nightly build, resulting in widespread and immediate data exfiltration²⁴. This incident highlights the fragility of relying on unverified external package repositories without strict hash-checking and internal mirroring.

5.2 Deserialization Code Execution and Metadata Parser Vulnerabilities

Historically, the AI community utilized the Python pickle module for serializing model weights to disk. However, pickle is inherently unsafe by design; an attacker can embed custom `__reduce__` methods into a model checkpoint, triggering arbitrary OS command execution the precise moment the model is loaded into memory via `torch.load()`²⁶. This exact mechanism was successfully leveraged in several high-profile CVEs, including CVE-2024-12029 (affecting InvokeAI servers with a CVSS of 9.8), CVE-2025-23298 (affecting NVIDIA Merlin Transformers4Rec), and CVE-2024-15969 (an unauthenticated RCE in the SGLang LoRA adapter loading endpoint)²⁶.

To mitigate pickle vulnerabilities, the industry rightfully shifted toward the safetensors format. Safetensors strictly stores deterministic tensor data as a flat dictionary of arrays, mathematically preventing code execution during the deserialization of the weights²⁸. While

safetensors successfully secures the binary weight data, it does not secure the accompanying configuration files, inadvertently shifting the primary attack vector directly to the metadata parsers²⁶.

Modern AI pipelines dynamically initialize complex software classes based on configuration metadata (e.g., `tokenizer_config.json`, `model_config.yaml`, or adapter configs defining the `base_model_name_or_path`)²⁹. The `hydra.utils.instantiate()` function, frequently utilized in frameworks like Hugging Face, imports and executes any dotted `_target_` specified in a configuration object²⁶. Therefore, an attacker distributing a perfectly secure safetensors weight file can simultaneously bundle a poisoned metadata file²⁹. When the victim's framework processes this metadata, it automatically resolves the application-level imports (e.g., executing `nemo.core.classes.common.os.system` which maps directly to standard POSIX shell commands), resulting in full RCE without ever requiring a malicious pickle file²⁶. Furthermore, metadata parsers relying on `ast.literal_eval` (such as FlexTok) can be targeted by stringified metadata bombs, resulting in immediate Denial of Service (DoS) through CPU and memory blowups before the model even begins to initialize²⁶.

5.3 Secure Architectural Contrasts: The franken_whisper Paradigm

To demonstrate that these vulnerabilities are architectural choices rather than unavoidable necessities, forensic auditors look to deterministic, memory-safe implementations. The `franken_whisper` project provides a vital contrast²⁸. Built entirely as a single Rust binary, it wraps Whisper backend engines behind a unified interface, utilizing pure Rust ggml model parsing and eliminating all Python Foreign Function Interfaces (FFI), subprocesses, and pickle dependencies²⁸.

The library rigorously freezes and verifies a deterministic 200-tensor safetensors package, enforcing exact package hashes and fail-closed numerical tolerances against compiled trust roots (e.g., verifying the SHA-256 digest `9276a840c...`)²⁸. It features cooperative cancellation propagating via cancellation tokens, real-time NDJSON streaming with stable schema validation (preventing regex parsing failures), and bounds-checked audio decoding via `symphonia` to prevent buffer overflows common in exotic `ffmpeg` codecs²⁸. By engineering the inference pipeline in bounded safe-Rust with explicit CPU kernels, the architecture proves that total forensic auditability and security are highly achievable when eliminating dynamic Python execution from the production path²⁸.

6. Forensic Attribution, Data Poisoning, and Influence Mathematics

When a deployed model exhibits malicious behavior—such as generating backdoor triggers, outputting factual inaccuracies, or executing poisoned logic—auditors must possess the mathematical capability to definitively trace the anomalous output back to specific poisoned data points within the multi-terabyte training corpus³⁰. Establishing causality requires rigorous mathematical frameworks for data attribution.

6.1 Classical Influence Functions and iHVP Matrix Limitations

Traditionally, the gold standard for attribution was leave-one-out (LOO) validation, which measures a data point's impact by repeatedly retraining the model with specific samples excluded³³. For modern Large Language Models possessing billions of parameters, LOO is computationally impossible, forcing reliance on gradient-based estimators³¹.

Classical Influence Functions (IF), derived from robust statistics, offer an analytical method to approximate the effect of a training data point on a test prediction by utilizing a first-order Taylor approximation, completely circumventing the need for model retraining³⁰. However, the core mathematical operation of classical IF requires the computation and inversion of the empirical risk Hessian matrix to produce Inverse-Hessian Vector Products (iHVP)³¹.

Forensic audits empirically demonstrate that classical influence functions are fundamentally inaccurate when applied to highly non-convex, high-dimensional LLM architectures³⁰. The approximation errors inherent in estimating the iHVP scale terribly; a conventional full Hessian

inversion requires a computational complexity of $O(Np^2 + p^3)$, where N is the dataset size and p represents the total number of parameters³⁶. While optimization techniques can

theoretically reduce this computational cost to $O(Np)$, the resulting mathematical fragility yields numerically unstable, inaccurate values that fail entirely to capture the true behavioral shifts of the language model³². Most fundamentally, influence scores measure parameter gradient shifts, which do not correlate linearly with complex semantic changes in generated text, rendering classical IF practically useless for deep forensic audits³³.

6.2 Data Shapley and Rescaled Influence Functions (RIF)

To achieve deterministic forensic attribution in high-dimensional spaces, auditors must implement advanced data valuation techniques. Data Shapley, derived from cooperative game theory, provides an equitable, theoretically grounded valuation of a data point's marginal contribution across all possible permutations and subsets of the training data³⁵. While theoretically pure, computing exact Shapley values remains highly resource-intensive, prompting the development of low-rank gradient approximations (LoGra) and LESS methodologies utilizing Low-Rank Adaptation (LoRA)³¹.

To bridge the gap, Rescaled Influence Functions (RIF) have emerged as the standard alternative. RIF incorporates limited second-order information by leveraging leave-one-out Hessian adjustments, retaining the additivity and computational scalability of a standard IF while vastly improving accuracy³⁰. By operating directly in the model's activation space and analyzing representation gradients, RIF provides a semantically meaningful signal, allowing auditors to trace generated outputs directly back to the specific poisoned document or malicious token injection at both the sample-level and fine-grained token-level³⁰. The successful deployment of RIF enables the precise excision of contaminated vectors, efficiently restoring pre-contamination model performance while maintaining deep provenance through

verifiable hash chains³⁵.

7. Regulatory Enforcement, Evidence Preservation, and Policy Directives

Securing this interconnected infrastructure requires strict adherence to standardized frameworks designed specifically to govern the AI software development lifecycle (SDLC), govern post-deployment runtime environments, and preserve digital evidence during forensic investigations.

7.1 NIST SP 800-218A: Secure Software Development Framework (SSDF)

In support of Executive Order (EO) 14110 (Safe, Secure, and Trustworthy Development and Use of Artificial Intelligence), the National Institute of Standards and Technology (NIST) released Special Publication 800-218A, establishing the authoritative Community Profile for Secure Software Development Practices for Generative AI and Dual-Use Foundation Models³⁸. The framework mandates that security cannot be appended as a cosmetic layer post-deployment; it must be deeply embedded into requirements, data sourcing, model architecture, and runtime orchestration³⁸.

The forensic mapping of SP 800-218A highlights critical control handoffs between development producers and deployment operators:

SP 800-218A Task	Framework Relevance	Deployment Control Mapping
PO.1.2 (Define AI software security requirements)	High	Deployment-time security requirements (e.g., input/output guardrails, continuous monitoring) must be explicitly specified during the initial development phase to prevent production gaps ⁴⁰ .
PS.1.1 (Protect code, models, and weights)	High	Model weight protection during development maps directly to rigorous access controls on model endpoints and infrastructure at deployment, strictly

		prohibiting insecure 0.0.0.0 bindings ⁴⁰ .
PW.1.1 (AI-specific threat modeling)	High	Development-time threat models must inform deployment-time infrastructure controls, enforcing guardrails at the API gateway layer rather than relying solely on the model's internal prompt adherence ⁴⁰ .
Supply Chain Integrity (SC-01 through SC-08)	High	CI/CD pipeline integrity requires automated dependency scanning, verifiable source provenance, and strict cryptographic hashing to defeat dependency confusion attacks (e.g., torchtriton) ³⁸ .

7.2 Forensic Volatility and Incident Response (NIST SP 800-86)

During an active infrastructure breach—such as detecting a ShadowRay botnet payload or a DCSync attack targeting an Active Directory connected to the AI cluster—incident responders must adhere strictly to the Order of Volatility defined in RFC 3227 and NIST SP 800-86⁴¹.

Evidence must be collected sequentially, prioritizing the most ephemeral data.

Responders must begin by acquiring CPU registers, routing tables, and ARP caches, proceeding systematically toward system memory (RAM), and concluding with non-volatile physical disk imaging using tools that preserve evidence integrity through MD5/SHA hash verification⁴¹. Volatile acquisition is paramount in AI systems, as in-memory state data, transient activation gradients, and ephemeral Docker orchestration containers represent the only cryptographic proof of active model poisoning or reverse shell execution before the infrastructure is wiped or reset via an XID failure³¹.

8. Conclusion: The Deterministic Causal Chain of Infrastructure Corruption

This forensic technical audit categorically reveals that the root causes of systemic violations and corruption in United States AI infrastructure are not isolated anomalies, nor are they the

result of emergent, uncontrollable AI agency. They are deeply interconnected, deterministic vulnerabilities spanning the entire computational stack.

The causal chain of corruption is highly visible:

1. **Hardware Fallibility:** The myth of silicon perfection is shattered by SDCs at the hyperscaler level, which inject mathematical poison into base gradient calculations⁴. Simultaneously, deterministically flawed proprietary firmware (evidenced by GSP watchdog timeouts resulting in XID 62 and XID 79) forces emergency bus drops, creating bare-metal denial-of-service conditions⁷.
2. **Topological Fragility:** When hardware fails silently, static interconnect protocols like NCCL generate ghost topologies¹². The network blindly polls non-existent endpoints, while improper PFC implementations in RoCEv2 environments induce cascading, network-wide deadlocks that degrade training efficiency¹².
3. **Exploitable Orchestration:** A degraded and fragile network is further compromised by inherently insecure default configurations in execution frameworks. By deploying tools like Ray without Job API authentication, administrators leave the infrastructure entirely exposed to automated ShadowRay botnet propagation and severe token exfiltration¹⁹.
4. **Supply Chain Execution:** Inside these compromised clusters, attackers bypass advanced binary protections like safetensors by injecting malicious RCE payloads directly into configuration metadata, exploiting dynamic instantiation mechanisms inherent in Python-based pipelines²⁶.
5. **Attribution Failure:** Finally, the ability to identify these precise failure chains is severely hampered by reliance on computationally flawed classical influence functions. True forensic accountability requires the adoption of Rescaled Influence Functions (RIF) and Data Shapley models, providing the exact mathematical precision required to trace a poisoned semantic output back to its physical data source³⁰.

In summation, AI infrastructure fails deterministically. Anthropomorphizing these systems serves only to obfuscate human engineering negligence. True security and compliance require uncompromising architectural isolation, stringent execution of the NIST SP 800-218A software development lifecycle, and an auditing methodology that maps the physical state of the hardware directly to the semantic output of the neural network.

Источники

1. Цена всемогущества_ одиночество творца.pdf
2. Silent Data Corruption: Advancing Detection, Diagnosis, and Mitigation Strategies, https://www.researchgate.net/publication/392564752_Silent_Data_Corruption_Advancing_Detection_Diagnosis_and_Mitigation_Strategies
3. The Dark Side of Computing: Silent Data Corruptions, <https://www.computer.org/csdl/magazine/co/2025/06/11014262/26WXZ2S290c>
4. Understanding Silent Data Corruptions in a Large Production CPU Population, https://www.researchgate.net/publication/374920382_Understanding_Silent_Data_Corruptions_in_a_Large_Production_CPU_Population
5. Silent Data Corruption in Artificial Intelligence: A Growing Challenge for

- Large-Scale Machine Learning - IEEE Computer Society,
<https://www.computer.org/csdl/magazine/mi/2026/01/11311359/2cEUqE5eDsY>
6. RTX PRO 6000 Blackwell (GB202) - Recurring full chip reset during sustained LLM inference, requires PSU power cycle to recover - NVIDIA Developer Forums,
<https://forums.developer.nvidia.com/t/rtx-pro-6000-blackwell-gb202-recurring-full-chip-reset-during-sustained-llm-inference-requires-psu-power-cycle-to-recover/365739>
 7. [Bug] Xid 62 GSP Watchdog Timeout on RTX 5080 during high-context LLM inference,
<https://forums.developer.nvidia.com/t/bug-xid-62-gsp-watchdog-timeout-on-rtx-5080-during-high-context-llm-inference/371272>
 8. RTX 4070 (AD104) GSP firmware crash (Xid 120 @ pc:0x1a92c96) under sustained CUDA workload — Windows BSOD + Linux GPU reset - NVIDIA Developer Forums,
<https://forums.developer.nvidia.com/t/rtx-4070-ad104-gsp-firmware-crash-xid-120-pc-0x1a92c96-under-sustained-cuda-workload-windows-bsod-linux-gpu-reset/369685>
 9. GSP-RM firmware bug: heartbeat stops after GC6 exit, leads to fatal GPU loss (Xid 79) and broken display recovery - NVIDIA Developer Forums,
<https://forums.developer.nvidia.com/t/gsp-rm-firmware-bug-heartbeat-stops-after-gc6-exit-leads-to-fatal-gpu-loss-xid-79-and-broken-display-recovery/366591>
 10. Xid 119 GSP Timeout on RTX 6000 Pro Blackwell (575.64.3) under Load – Reproducible Crash - NVIDIA Developer Forums,
<https://forums.developer.nvidia.com/t/xid-119-gsp-timeout-on-rtx-6000-pro-blackwell-575-64-3-under-load-reproducible-crash/337871>
 11. RTX PRO 6000 Blackwell — Xid 79 + Xid 154 GSP crash under vLLM load #46 - GitHub, <https://github.com/dbirks/home-k8s/issues/46>
 12. The Ghost in the Datacenter: Link Flapping, Topology Knowledge Failures, and the FITO Category Mistake - arXiv, <https://arxiv.org/html/2603.03736v1>
 13. The DMA Streaming Framework: Kernel-Level Buffer Orchestration for High-Performance AI Data Paths - arXiv, <https://arxiv.org/pdf/2603.10030>
 14. The DMA Streaming Framework: Kernel-Level Buffer Orchestration for High-Performance AI Data Paths - arXiv, <https://arxiv.org/html/2603.10030v1>
 15. Scale Up Ethernet Framework Overview | PDF - Scribd,
<https://www.scribd.com/document/947671189/Scale-Ethernet-RM102>
 16. Scale-Up Ethernet Framework Specification - Support Documents and Downloads, <https://docs.broadcom.com/docs/scale-up-ethernet-framework>
 17. ShadowRay, Campaign C0045 | MITRE ATT&CK®,
<https://attack.mitre.org/campaigns/C0045/>
 18. Two-Year-Old Ray AI Framework Flaw Exploited in Ongoing Campaign - SecurityWeek,
<https://www.securityweek.com/two-year-old-ray-ai-framework-flaw-exploited-in-ongoing-campaign/>
 19. CVE-2023-48022 Impact, Exploitability, and Mitigation Steps | Wiz,
<https://www.wiz.io/vulnerability-database/cve/cve-2023-48022>

20. CVE-2023-48022: Anyscale Ray RCE Vulnerability - SentinelOne, <https://www.sentinelone.com/vulnerability-database/cve-2023-48022/>
21. ShadowRay: First Known Attack Campaign Targeting AI Workloads Actively Exploited In The Wild - Oligo Security, <https://www.oligo.security/blog/shadowray-attack-ai-workloads-actively-exploited-in-the-wild>
22. ShadowRay 2.0: Self-replicating botnet turns Ray clusters into DDoS weapons - FastNetMon, <https://fastnetmon.com/2025/11/21/shadowray-2-0-self-replicating-botnet-turns-ray-clusters-into-ddos-weapons/>
23. Runtime Security Becomes the Source of Truth: Oligo's 2025 Breakthrough Year, <https://www.oligo.security/blog/reflecting-on-a-breakthrough-year>
24. Killing Two Birds with One Stone: Malicious Package Detection in NPM and PyPI using a Single Model of Malicious Behavior Sequence - arXiv, <https://arxiv.org/html/2309.02637v2>
25. PyTorch dependency 'torchtriton' Supply Chain Attack - SentinelOne, <https://www.sentinelone.com/blog/pytorch-dependency-torchtriton-supply-chain-attack/>
26. AI Models RCE - HackTricks, <https://hacktricks.wiki/en/AI/AI-Models-RCE.html>
27. AI Security Timeline - Karan Bansal | CVEs, Breaches & Exploits in the AI Stack, <https://karanbansal.in/ai-security-timeline/>
28. FRANKEN_WHISPER — Rust utility // Lib.rs, https://lib.rs/crates/franken_whisper
29. terminal-bench-core==head, <https://www.tbench.ai/registry/terminal-bench-core/head>
30. Rescaled Influence Functions: Accurate Data Attribution in High Dimension | OpenReview, <https://openreview.net/forum?id=OT9IN3m3pc-eld=p3Gr6WJNKw>
31. Where Did It Go Wrong? Attributing Undesirable LLM Behaviors via Representation Gradient Tracing - arXiv, <https://arxiv.org/html/2510.02334v1>
32. Where did it go wrong? Attributing undesirable LLM behaviors via representation gradient tracing - InK@SMU.edu.sg, https://ink.library.smu.edu.sg/cgi/viewcontent.cgi?article=12194&context=sis_research
33. Do Influence Functions Work on Large Language Models? - arXiv, <https://arxiv.org/html/2409.19998v2>
34. (PDF) Training data influence analysis and estimation: a survey - ResearchGate, https://www.researchgate.net/publication/379410061_Training_data_influence_analysis_and_estimation_a_survey
35. Understanding Black-box Predictions via Influence Functions - ResearchGate, https://www.researchgate.net/publication/315111148_Understanding_Black-box_Predictions_via_Influence_Functions
36. Adversarial Inference and Manipulation of Machine Learning Models - Publikationen der UdS - Universität des Saarlandes, https://publikationen.sulb.uni-saarland.de/bitstream/20.500.11880/39930/1/Disser-tation_RuiWen.pdf
37. LoRIF: Low-Rank Influence Functions for Scalable Training Data Attribution - arXiv,

<https://arxiv.org/pdf/2601.21929>

38. Secure Software Development Practices for Generative AI and Dual-Use Foundation Models: An SSDF Community Profile | Request PDF - ResearchGate, https://www.researchgate.net/publication/382801536_Secure_Software_Development_Practices_for_Generative_AI_and_Dual-Use_Foundation_Models_An_SSDF_Community_Profile
39. SP 800-218A, Secure Software Development Practices for Generative AI and Dual-Use Foundation Models: An SSDF Community Profile | CSRC, <https://csrc.nist.gov/pubs/sp/800/218/a/final>
40. NIST SP 800-218A Mapping - AI Secured by Design, <https://aisecuredbydesign.io/infrastructure/mappings/nist-sp800-218a/>
41. Anthropic-Cybersecurity-Skills/index.json at main - GitHub, <https://github.com/mukul975/Anthropic-Cybersecurity-Skills/blob/main/index.json>
42. collecting-volatile-evidence-from-co... · LobeHub, <https://lobehub.com/pl/skills/chehdung94-vietnamese-banking-cyber-skills-collecting-volatile-evidence-from-compromised-host>